Spring Framework

Interview Questions & Answers

50 Questions – Complete Guide with Definitions & Examples

Spring Core • IoC & DI • AOP • Spring MVC

Spring Boot • REST API • Security • Transactions

Microservices • Profiles • Testing • CORS & JWT

SECTION 1 – Spring Core Concepts

1 What is the Spring Framework? Why is it used?

Definition

Spring Framework is an open-source, lightweight Java framework for building enterprise applications. It provides comprehensive infrastructure support — IoC, DI, AOP, MVC, Data, Security — letting developers focus on business logic instead of boilerplate code.

- **IoC Container** – Manages object creation and lifecycle.
- **Dependency Injection** – Wires dependencies automatically.
- **AOP** – Cross-cutting concerns (logging, security) without modifying code.
- **Spring MVC** – Web framework built on DispatcherServlet pattern.
- **Spring Data** – Simplified database access (JDBC, JPA, NoSQL).
- **Spring Security** – Authentication and authorisation framework.
- **Spring Boot** – Auto-configured, opinionated extension of Spring.

Example

```
// Without Spring – manual wiring
OrderService svc = new OrderService(new PaymentService(new PaymentRepo()));

// With Spring – container wires everything
@Service
class OrderService {
private final PaymentService paymentService;
public OrderService(PaymentService paymentService) {
this.paymentService = paymentService; // injected by Spring
}
}
```

2 Explain Inversion of Control (IoC) and Dependency Injection (DI)

Definition

IoC is a design principle where control of object creation is transferred from the application code to the Spring container. DI is the mechanism Spring uses to implement IoC — it injects dependencies into a class rather than the class creating them itself.

- **IoC**: 'Don't call us, we'll call you.' The framework creates and manages objects.
- **DI**: The container injects required dependencies through constructor, setter, or field.
- **Benefits**: Loose coupling, easier testing (inject mocks), better separation of concerns.

Example

```
// Without IoC/DI – tight coupling
class OrderService {
PaymentService ps = new PaymentService(); // hard-coded dependency
}

// With IoC/DI – loose coupling
@Service
class OrderService {
private final PaymentService ps; // Spring injects this
public OrderService(PaymentService ps) { this.ps = ps; }
}
```

3 What are the different types of Dependency Injection in Spring?

Definition

Spring supports three types of DI, each suited for different scenarios.

- **Constructor Injection** – Dependencies passed via constructor. Recommended: immutable, required dependencies.
- **Setter Injection** – Dependencies set via setter methods. Use for optional dependencies.
- **Field Injection** – @Autowired directly on field. Convenient but not recommended (hard to test, hidden dependencies).

Example

```
// 1. Constructor Injection (RECOMMENDED)
@Service
class OrderService {
private final PaymentService paymentService;
public OrderService(PaymentService paymentService) {
this.paymentService = paymentService;
}
}
// 2. Setter Injection (optional dependencies)
@Service
class ReportService {
private EmailService emailService;
@Autowired
public void setEmailService(EmailService es) { this.emailService = es; }
}
// 3. Field Injection (avoid in production)
@Service
class UserService {
@Autowired
private UserRepository userRepo; // directly injected
}
```

4 What is the difference between ApplicationContext and BeanFactory?

Definition

Both are Spring IoC containers. ApplicationContext is the advanced, recommended container. BeanFactory is the basic, lightweight container.

Example

```
// BeanFactory – basic, lazy initialization (avoid in most cases)
BeanFactory factory = new XmlBeanFactory(new ClassPathResource("beans.xml"));
MyBean bean = factory.getBean(MyBean.class);

// ApplicationContext – full-featured (use this always)
ApplicationContext ctx =
new AnnotationConfigApplicationContext(AppConfig.class);
MyBean bean = ctx.getBean(MyBean.class);

// Spring Boot creates ApplicationContext automatically:
SpringApplication.run(MyApp.class, args);
```

Note: ApplicationContext is a superset of BeanFactory. Always use ApplicationContext.

Feature	BeanFactory	ApplicationContext
Initialization	Lazy (on first getBean())	Eager (all singletons at startup)
AOP Support	No	Yes
Event Publishing	No	Yes (ApplicationEventPublisher)
Internationalization	No	Yes (MessageSource)
Annotation Support	No	Yes (@Autowired, @PostConstruct)
Environment/Profiles	No	Yes

5 Explain Spring Bean and its Lifecycle

Definition

A Spring Bean is any object managed by the Spring IoC container. The lifecycle covers object creation, dependency injection, initialisation, use, and destruction.

- **1. Instantiation** – Container creates the bean (calls constructor).
- **2. Populate Properties** – Dependencies are injected (DI happens here).
- **3. BeanNameAware / BeanFactoryAware** – Awareness interfaces called.
- **4. BeanPostProcessor (before)** – Pre-initialisation processing.
- **5. @PostConstruct / afterPropertiesSet()** – Custom initialisation logic.
- **6. BeanPostProcessor (after)** – Post-initialisation processing.
- **7. Bean ready for use** – Application uses the bean.
- **8. @PreDestroy / destroy()** – Cleanup before container shuts down.

Example

```
@Component
public class DatabasePool {
@PostConstruct // Step 5 – runs after injection
public void init() {
System.out.println("Opening database connections");
}
@PreDestroy // Step 8 – runs at shutdown
public void cleanup() {
System.out.println("Closing database connections");
}
}
```

6 What are the different scopes of Spring Beans?

Definition

Bean scope defines how many instances Spring creates and their lifecycle.

Example

```
// singleton – ONE instance per ApplicationContext (default)
@Component @Scope("singleton")
public class OrderService {}

// prototype – NEW instance every time it is requested
@Component @Scope("prototype")
public class ShoppingCart {}

// Web scopes
@Component @RequestScope // new instance per HTTP request
public class RequestLogger {}

@Component @SessionScope // new instance per HTTP session
public class UserPreferences {}

@Component @ApplicationScope // one per ServletContext
public class AppConfig {}
```

Scope	Instances	Lifecycle	Best For
singleton	1 per context	Container lifecycle	Stateless services, DAOs
prototype	New every request	Caller manages	Stateful objects
request	1 per HTTP request	HTTP request	Request-scoped data
session	1 per HTTP session	HTTP session	User session data
application	1 per ServletContext	App lifecycle	App-wide shared state

7 Role of @Component, @Service, @Repository, @Controller

Definition

All four are specialised stereotype annotations that register classes as Spring beans. They differ in semantic meaning and some provide extra behaviour.

- **@Component** – Generic Spring-managed bean. Base for all stereotypes.
- **@Service** – Business logic layer. No extra behaviour; improves readability.
- **@Repository** – DAO/persistence layer. Enables Spring's exception translation (maps DB exceptions to DataAccessException).
- **@Controller** – MVC controller. Handles HTTP requests and returns views.
- **@RestController** – @Controller + @ResponseBody. Returns JSON/XML directly.

Example

```
@Repository
public class EmployeeRepository {
// DataAccessException translation enabled here
}

@Service
public class EmployeeService {
@Autowired EmployeeRepository repo;
}

@RestController
@RequestMapping("/api/employees")
public class EmployeeController {
@Autowired EmployeeService svc;
@GetMapping public List getAll() { return svc.findAll(); }
}
```

8 What is @Autowired, @Resource, and @Inject – differences?

Definition

All three annotations inject dependencies but come from different specifications.

Example

```
// @Autowired (Spring-specific) – injects by TYPE, then by name
@Autowired
private PaymentService paymentService;

// @Autowired with @Qualifier – disambiguate when multiple beans exist
@Autowired @Qualifier("stripePayment")
private PaymentService paymentService;

// @Resource (Java EE / Jakarta EE) – injects by NAME, then by type
@Resource(name = "stripePayment")
private PaymentService paymentService;

// @Inject (JSR-330) – same as @Autowired but no 'required' attribute
@Inject
private PaymentService paymentService;

// RECOMMENDATION: Use @Autowired (Spring) or constructor injection
```

Annotation	Source	Inject By	required attr	Usage
@Autowired	Spring	Type then name	Yes	Recommended in Spring
@Resource	JSR-250	Name then type	No	When name-based injection needed
@Inject	JSR-330	Type	No	Framework-agnostic code

9 How does Spring handle circular dependencies?

Definition

A circular dependency occurs when Bean A depends on Bean B and Bean B depends on Bean A. Spring handles this differently based on injection type.

- **Constructor injection**: Spring CANNOT resolve circular dependencies — throws BeanCurrentlyInCreationException.
- **Setter/Field injection**: Spring CAN resolve — it creates both beans first (partially), then injects.
- **@Lazy**: Breaks the cycle by creating a proxy first and injecting the real bean lazily.

Example

```
// PROBLEM: Constructor circular dependency
@Service class A { public A(B b) {} }
@Service class B { public B(A a) {} }
// BeanCurrentlyInCreationException!

// SOLUTION 1: Use setter injection
@Service class A {
private B b;
@Autowired public void setB(B b) { this.b = b; }
}

// SOLUTION 2: @Lazy annotation
@Service class A {
private final B b;
public A(@Lazy B b) { this.b = b; } // proxy injected first
}

// SOLUTION 3: Redesign – extract shared logic to a third service
```

Note: Best solution: Redesign to avoid circular dependencies. They indicate a design smell.

10 What is Spring AOP? How is it different from OOP?

Definition

AOP (Aspect-Oriented Programming) complements OOP by separating cross-cutting concerns (logging, security, transactions) from business logic without modifying the business code.

- **OOP** – Organises code into classes and objects. Cannot cleanly separate cross-cutting concerns.
- **AOP** – Organises cross-cutting concerns into Aspects. Applied to target methods via Pointcuts.
- **Key Terms**: Aspect (module for cross-cutting), JoinPoint (method execution point), Advice (@Before/@After/@Around), Pointcut (expression selecting join points), Weaving (linking aspect to code).

Example

```
@Aspect
@Component
public class LoggingAspect {

// Pointcut: all methods in service package
@Pointcut("execution(* com.example.service.*.*(..))")
public void serviceMethods() {}

@Before("serviceMethods()")
public void logBefore(JoinPoint jp) {
System.out.println("Before: " + jp.getSignature().getName());
}

@AfterReturning(pointcut="serviceMethods()", returning="result")
public void logAfter(JoinPoint jp, Object result) {
System.out.println("Returned: " + result);
}

@Around("serviceMethods()")
public Object timeMethod(ProceedingJoinPoint pjp) throws Throwable {
long start = System.currentTimeMillis();
Object result = pjp.proceed();
System.out.println("Took: " + (System.currentTimeMillis()-start) + "ms");
return result;
}
}
```

11 JDK Dynamic Proxy vs CGLIB Proxy in Spring AOP

Definition

Spring AOP wraps target beans in proxies to apply Advice. Two proxy mechanisms exist.

- **JDK Dynamic Proxy**: Used when the target bean implements an interface. Creates a proxy that implements the same interface. Requires interface.
- **CGLIB Proxy**: Used when the target class does NOT implement an interface. Creates a subclass of the target at runtime. Cannot proxy final classes/methods.

Example

```
// JDK Proxy – target implements interface
public interface OrderService { Order placeOrder(OrderRequest req); }

@Service
public class OrderServiceImpl implements OrderService {
@Override public Order placeOrder(OrderRequest req) { ... }
}
// Spring creates proxy that also implements OrderService

// CGLIB Proxy – no interface
@Service
public class PaymentService { // no interface
public void processPayment(double amount) { ... }
}
// Spring creates a subclass of PaymentService as proxy

// Force CGLIB even with interface:
@EnableAspectJAutoProxy(proxyTargetClass = true)
```

12 How does Spring handle Transactions? @Transactional explained

Definition

Spring's transaction management wraps method execution in a database transaction. If the method completes successfully → commit. If RuntimeException → rollback.

Example

```
// Enable transaction management
@Configuration
@EnableTransactionManagement
public class AppConfig {
@Bean
public PlatformTransactionManager txManager(DataSource ds) {
return new DataSourceTransactionManager(ds);
}
}

// @Transactional on method – applies to that method only
@Service
public class TransferService {
@Transactional
public void transfer(Long from, Long to, double amount) {
accountRepo.debit(from, amount);
accountRepo.credit(to, amount);
// Rollback if EITHER operation throws RuntimeException
}

// readOnly optimisation for queries
@Transactional(readOnly = true)
public Account getAccount(Long id) {
return accountRepo.findById(id).orElseThrow();
}

// Custom rollback rules
@Transactional(rollbackFor = Exception.class,
noRollbackFor = ValidationException.class)
public void processOrder(Order order) { ... }
}

// @Transactional on class – all public methods are transactional
@Service
@Transactional
public class OrderService {
// every public method gets a transaction
@Transactional(readOnly = true) // override for reads
public List findAll() { ... }
}
```

13 Programmatic vs Declarative Transaction Management

Definition

Two ways to manage transactions in Spring.

- **Declarative** – Use @Transactional annotation. Spring AOP applies transactions automatically. Clean, no boilerplate. Recommended.
- **Programmatic** – Manually manage transactions using TransactionTemplate or PlatformTransactionManager. More control but verbose.

Example

```
// Declarative (Recommended)
@Transactional
public void placeOrder(Order order) {
orderRepo.save(order);
paymentService.charge(order);
}

// Programmatic – TransactionTemplate
@Service
public class OrderService {
private final TransactionTemplate txTemplate;
public OrderService(PlatformTransactionManager txm) {
this.txTemplate = new TransactionTemplate(txm);
}
public void placeOrder(Order order) {
txTemplate.execute(status -> {
try {
orderRepo.save(order);
paymentService.charge(order);
return null;
} catch (Exception e) {
status.setRollbackOnly(); // manual rollback
throw e;
}
});
}
}
```

14 What is @PostConstruct and @PreDestroy vs InitializingBean and DisposableBean?

Definition

Both provide bean lifecycle hooks but are used differently.

- **@PostConstruct** – Method runs after dependencies are injected. Part of JSR-250 (portable).
- **@PreDestroy** – Method runs before the bean is destroyed. For cleanup.
- **InitializingBean.afterPropertiesSet()** – Spring-specific init callback. Less preferred.
- **DisposableBean.destroy()** – Spring-specific destroy callback. Less preferred.

Example

```
// PREFERRED: @PostConstruct / @PreDestroy (portable)
@Component
public class CacheService {
private Map cache;

@PostConstruct // runs after @Autowired fields are set
public void initCache() {
cache = new ConcurrentHashMap<>();
System.out.println("Cache initialised");
}

@PreDestroy // runs before context closes
public void clearCache() {
cache.clear();
System.out.println("Cache cleared");
}
}

// Spring-specific (less preferred)
public class LegacyService implements InitializingBean, DisposableBean {
@Override public void afterPropertiesSet() { /* init */ }
@Override public void destroy() { /* cleanup */ }
}
```

15 What is Spring JDBC? How does it differ from Hibernate?

Definition

Spring JDBC (JdbcTemplate) is a thin wrapper over raw JDBC that eliminates boilerplate while keeping full control of SQL. Hibernate is a full ORM that auto-generates SQL.

Example

```
// Spring JDBC – you write SQL
@Repository
public class EmployeeDao {
@Autowired JdbcTemplate jdbc;

public Employee findById(Long id) {
return jdbc.queryForObject(
"SELECT * FROM employee WHERE id=?",
(rs, row) -> new Employee(
rs.getLong("id"), rs.getString("name")),
id);
}
public int save(Employee e) {
return jdbc.update(
"INSERT INTO employee(name,salary) VALUES(?,?)",
e.getName(), e.getSalary());
}
}

// Hibernate/JPA – SQL is auto-generated
public interface EmployeeRepository extends JpaRepository {
// No SQL needed – Spring Data generates all queries
}
```

Aspect	Spring JDBC	Hibernate/JPA
SQL Control	Full SQL control	SQL auto-generated
Learning Curve	Low	Higher (entities, mappings, JPQL)
Performance	Predictable	May produce N+1, lazy loading issues
Portability	Not DB-portable	DB-portable via dialects
Best For	Complex SQL, stored procedures	CRUD-heavy applications

SECTION 2 – Spring MVC

16 What is the DispatcherServlet? How does it work?

Definition

DispatcherServlet is the Front Controller in Spring MVC. All HTTP requests go through it, and it delegates to the appropriate handler (controller method).

- **Step 1** – Browser sends request to DispatcherServlet.
- **Step 2** – HandlerMapping finds which @Controller method handles this URL.
- **Step 3** – HandlerAdapter invokes the controller method.
- **Step 4** – Controller returns a Model and View name (MVC) or response body (REST).
- **Step 5** – ViewResolver resolves the view name to a template (Thymeleaf, JSP).
- **Step 6** – View renders the response and sends back to the browser.

Example

```
// Auto-configured in Spring Boot (no XML needed)
// Mapped to '/' by default (all requests)

// Controller handles specific URL
@Controller
public class HomeController {
@GetMapping("/home")
public String home(Model model) {
model.addAttribute("msg", "Welcome!");
return "home"; // ViewResolver -> templates/home.html
}
}

// application.properties
spring.mvc.view.prefix=/WEB-INF/views/
spring.mvc.view.suffix=.jsp
```

17 Explain HandlerInterceptor in Spring MVC

Definition

HandlerInterceptor is similar to a Servlet Filter but specific to Spring MVC. It intercepts requests before and after controller execution.

- **preHandle()** – Called before the controller. Return false to stop processing.
- **postHandle()** – Called after controller but before view rendering.
- **afterCompletion()** – Called after the complete request (even on exception). For cleanup.

Example

```
@Component
public class AuthInterceptor implements HandlerInterceptor {

@Override
public boolean preHandle(HttpServletRequest req,
HttpServletResponse res, Object handler) {
String token = req.getHeader("Authorization");
if (token == null) {
res.setStatus(401);
return false; // STOP – don't call controller
}
return true; // continue
}

@Override
public void postHandle(HttpServletRequest req, HttpServletResponse res,
Object handler, ModelAndView mav) {
// Add common model attributes
}

@Override
public void afterCompletion(HttpServletRequest req, HttpServletResponse res,
Object handler, Exception ex) {
// Cleanup resources
}
}

// Register the interceptor
@Configuration
public class WebConfig implements WebMvcConfigurer {
@Override
public void addInterceptors(InterceptorRegistry registry) {
registry.addInterceptor(new AuthInterceptor())
.addPathPatterns("/api/**")
.excludePathPatterns("/api/auth/**");
}
}
```

18 What is @Controller vs @RestController?

Definition

@Controller is for traditional MVC (returns views). @RestController is for REST APIs (returns JSON/XML).

Example

```
// @Controller – returns view name (HTML page)
@Controller
public class PageController {
@GetMapping("/dashboard")
public String dashboard(Model model) {
model.addAttribute("user", currentUser);
return "dashboard"; // templates/dashboard.html
}
// Can also return JSON with @ResponseBody:
@GetMapping("/api/status")
@ResponseBody
public String status() { return "OK"; }
}

// @RestController = @Controller + @ResponseBody on ALL methods
@RestController
@RequestMapping("/api/products")
public class ProductController {
@GetMapping("{id}")
public Product get(@PathVariable Long id) {
return productService.findById(id); // auto-serialised to JSON
}
}
```

19 How to enable CORS in a Spring MVC application?

Definition

CORS (Cross-Origin Resource Sharing) allows browsers on a different origin (domain/port) to call your API. Spring provides several ways to enable CORS.

Example

```
// Method 1: @CrossOrigin on a controller
@RestController
@CrossOrigin(origins = "http://localhost:3000", maxAge = 3600)
public class ProductController { ... }

// Method 2: Global CORS in WebMvcConfigurer
@Configuration
public class WebConfig implements WebMvcConfigurer {
@Override
public void addCorsMappings(CorsRegistry registry) {
registry.addMapping("/api/**")
.allowedOrigins("http://localhost:3000","https://myfrontend.com")
.allowedMethods("GET","POST","PUT","DELETE","OPTIONS")
.allowedHeaders("*")
.allowCredentials(true)
.maxAge(3600);
}
}

// Method 3: In Spring Security (with Spring Boot)
http.cors(cors -> cors.configurationSource(corsConfigSource()));

@Bean CorsConfigurationSource corsConfigSource() {
CorsConfiguration config = new CorsConfiguration();
config.setAllowedOrigins(List.of("http://localhost:3000"));
config.setAllowedMethods(List.of("GET","POST","PUT","DELETE"));
UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
source.registerCorsConfiguration("/**", config);
return source;
}
```

20 How to define a custom scope for a Spring bean?

Definition

You can create completely custom scopes by implementing the Scope interface.

Example

```
// Custom scope implementation
public class TenantScope implements Scope {
private final Map> tenantBeans
= new ConcurrentHashMap<>();

@Override
public Object get(String name, ObjectFactory factory) {
String tenantId = TenantContext.getCurrentTenant();
return tenantBeans
.computeIfAbsent(tenantId, k -> new ConcurrentHashMap<>())
.computeIfAbsent(name, k -> factory.getObject());
}
@Override public Object remove(String name) { return null; }
@Override public String getConversationId() {
return TenantContext.getCurrentTenant();
}
@Override public void registerDestructionCallback(
String name, Runnable callback) {}
@Override public Object resolveContextualObject(String key) { return null; }
}

// Register the custom scope
@Configuration
public class ScopeConfig implements BeanFactoryPostProcessor {
@Override
public void postProcessBeanFactory(ConfigurableBeanFactory bf) {
bf.registerScope("tenant", new TenantScope());
}
}

// Use the custom scope
@Component
@Scope("tenant")
public class TenantConfig { ... }
```

SECTION 3 – Spring Boot

21 What is Spring Boot? How is it different from Spring Framework?

Definition

Spring Boot is an opinionated, convention-over-configuration extension of Spring Framework that auto-configures everything based on classpath and eliminates boilerplate setup.

- **Spring Framework**: Requires manual configuration of every bean, DataSource, MVC, Security etc.
- **Spring Boot**: Auto-configures all of that. Just add starters and it works.
- **Embedded Server**: Spring Boot ships with Tomcat; deploy as a JAR, not WAR.
- **Starter POMs**: Curated dependency sets (spring-boot-starter-web includes Tomcat, MVC, Jackson).
- **Actuator**: Production monitoring out of the box (/health, /metrics).

Example

```
// Spring Framework – requires manual DataSource, TransactionManager, MVC config...
// ...hundreds of lines of XML or @Configuration code

// Spring Boot – zero configuration needed
@SpringBootApplication
public class App {
public static void main(String[] args) {
SpringApplication.run(App.class, args);
}
}
// Just add spring-boot-starter-web and it configures Tomcat, MVC, Jackson
```

22 What is @SpringBootApplication vs @EnableAutoConfiguration?

Definition

@SpringBootApplication is a meta-annotation that includes @EnableAutoConfiguration plus two others.

- **@SpringBootApplication** = @SpringBootConfiguration + @EnableAutoConfiguration + @ComponentScan.
- **@EnableAutoConfiguration** – Triggers Spring Boot's auto-configuration mechanism only.
- **@SpringBootConfiguration** – Same as @Configuration (marks as config class).
- **@ComponentScan** – Scans current package and sub-packages for components.

Example

```
// @SpringBootApplication (use this – combines all three)
@SpringBootApplication
public class App { public static void main(String[] args) {...} }

// Equivalent to:
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan(basePackages = "com.example")
public class App { public static void main(String[] args) {...} }

// Exclude specific auto-configuration
@SpringBootApplication(exclude = {
DataSourceAutoConfiguration.class,
SecurityAutoConfiguration.class
})
public class App { ... }
```

23 How does Spring Boot auto-configuration work internally?

Definition

Spring Boot reads a list of auto-configuration classes and applies them conditionally based on what is on the classpath and what beans are already defined.

- **Step 1**: Spring Boot reads META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports from every JAR.
- **Step 2**: Each listed class has @Conditional annotations: @ConditionalOnClass, @ConditionalOnMissingBean, @ConditionalOnProperty.
- **Step 3**: If conditions pass, the @Configuration class is applied and its @Bean methods register beans.
- **User beans override auto-configured beans**: If you define your own DataSource @Bean, Spring Boot won't auto-configure one.

Example

```
// Example: DataSourceAutoConfiguration (simplified)
@Configuration
@ConditionalOnClass({ DataSource.class, EmbeddedDatabase.class })
@ConditionalOnMissingBean(DataSource.class) // only if no custom DataSource
@EnableConfigurationProperties(DataSourceProperties.class)
public class DataSourceAutoConfiguration {
@Bean
@ConditionalOnProperty(name="spring.datasource.url")
public DataSource dataSource(DataSourceProperties props) {
return DataSourceBuilder.create()
.url(props.getUrl())
.username(props.getUsername())
.password(props.getPassword())
.build();
}
}

# Debug auto-configuration at startup:
debug=true # shows CONDITIONS EVALUATION REPORT
```

24 What is the spring.factories file?

Definition

spring.factories (Spring Boot 2.x) / AutoConfiguration.imports (Spring Boot 3.x) is a file that lists all auto-configuration classes for a JAR. Spring Boot reads this file at startup to discover what auto-configurations to apply.

Example

```
# Spring Boot 2.x: META-INF/spring.factories
org.springframework.boot.autoconfigure.EnableAutoConfiguration=\
com.example.MyAutoConfiguration,\
org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration,\
org.springframework.boot.autoconfigure.web.servlet.WebMvcAutoConfiguration

# Spring Boot 3.x: META-INF/spring/
# org.springframework.boot.autoconfigure.AutoConfiguration.imports
com.example.MyAutoConfiguration
org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration

// Your custom AutoConfiguration class
@AutoConfiguration
@ConditionalOnClass(SmsService.class)
@ConditionalOnMissingBean(SmsService.class)
public class SmsAutoConfiguration {
@Bean
public SmsService smsService() { return new TwilioSmsService(); }
}
```

25 Purpose of application.properties and application.yml?

Definition

These are Spring Boot's main configuration files loaded automatically at startup.

Example

```
# application.properties format
server.port=8080
spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
spring.datasource.username=admin
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update

# application.yml format (hierarchical, preferred)
server:
port: 8080
spring:
datasource:
url: jdbc:postgresql://localhost:5432/mydb
username: admin
jpa:
show-sql: true
hibernate:
ddl-auto: update

# Custom properties
app:
jwt-secret: mySecretKey
max-retries: 3

// Inject custom properties:
@Value("${app.jwt-secret}")
private String jwtSecret;

// Or with @ConfigurationProperties:
@ConfigurationProperties(prefix = "app")
public class AppProperties {
private String jwtSecret;
private int maxRetries;
}
```

26 How does Spring Boot handle dependency management?

Definition

Spring Boot uses a Bill of Materials (BOM) through spring-boot-starter-parent to manage compatible dependency versions automatically.

Example

```
<!-- pom.xml – Spring Boot parent manages ALL versions -->

org.springframework.boot
spring-boot-starter-parent
3.2.0

<!-- No version needed – parent manages it -->

org.springframework.boot
spring-boot-starter-web
<!-- version inherited from parent -->

org.springframework.boot
spring-boot-starter-data-jpa

<!-- Override a version if needed -->

2.16.0
```

27 What are Spring Boot Starters?

Definition

Starters are pre-packaged dependency sets for specific concerns. Adding one starter brings in all required libraries at compatible versions.

- **spring-boot-starter-web** – Spring MVC + Tomcat + Jackson.
- **spring-boot-starter-data-jpa** – Spring Data JPA + Hibernate + JDBC.
- **spring-boot-starter-security** – Spring Security.
- **spring-boot-starter-test** – JUnit 5 + Mockito + AssertJ + MockMvc.
- **spring-boot-starter-actuator** – Health + metrics endpoints.
- **spring-boot-starter-cache** – Caching abstraction.
- **spring-boot-starter-mail** – JavaMail for email sending.

Example

```
<!-- Just add one starter and everything is configured -->

org.springframework.boot
spring-boot-starter-web
<!-- Includes: spring-web, spring-webmvc, tomcat-embed-core,
jackson-databind, hibernate-validator, logback -->

<!-- Automatically creates:
- Embedded Tomcat server
- DispatcherServlet
- Jackson ObjectMapper
- Default error handling -->
```

28 How does Spring Boot handle embedded servers?

Definition

Spring Boot packages an embedded web server (Tomcat by default) inside the JAR, so no external server installation is needed.

- **Default**: Tomcat (via spring-boot-starter-web).
- **Switch to Jetty**: Exclude Tomcat, add spring-boot-starter-jetty.
- **Switch to Undertow**: Exclude Tomcat, add spring-boot-starter-undertow.
- **Configuration**: server.port, server.ssl.*, server.servlet.context-path etc.

Example

```
<!-- Switch from Tomcat to Jetty -->

org.springframework.boot
spring-boot-starter-web

spring-boot-starter-tomcat


spring-boot-starter-jetty

# Server configuration
server.port=9090
server.servlet.context-path=/myapp
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-password=secret
server.ssl.key-store-type=PKCS12
server.shutdown=graceful # wait for in-flight requests
```

29 Different ways to configure a Spring Boot application?

Definition

Spring Boot offers multiple configuration sources in priority order.

- **application.properties / application.yml** – Most common. In src/main/resources.
- **Profile-specific files** – application-dev.properties, application-prod.yml.
- **Environment variables** – SERVER_PORT=9090 (overrides properties).
- **Command-line arguments** – java -jar app.jar --server.port=9090 (highest priority).
- **@ConfigurationProperties** – Bind a prefix of properties to a POJO.
- **@Value** – Inject individual property values.
- **Config Server** – Spring Cloud Config for centralised configuration.

Example

```
# Priority (highest to lowest):
# 1. Command-line: --server.port=9090
# 2. Environment variables: SERVER_PORT=9090
# 3. application-{profile}.properties outside JAR
# 4. application.properties outside JAR
# 5. application-{profile}.properties inside JAR
# 6. application.properties inside JAR
```

```
@ConfigurationProperties(prefix = "app.mail")
@Component
public class MailProperties {
private String host;
private int port = 587;
private String username;
private String password;
// getters/setters
}
```

```
# application.yml
app:
mail:
host: smtp.gmail.com
port: 587
username: user@gmail.com
```

30 What is Spring Boot DevTools?

Definition

Spring Boot DevTools is a development-time tool that improves the developer experience with automatic restarts and live reload.

- **Automatic Restart** – Application restarts when classpath files change (faster than full restart).
- **LiveReload** – Browser auto-refreshes when resources change (needs LiveReload browser extension).
- **Disabled caching** – Template caching disabled in dev for immediate reflection of changes.
- **Remote debugging** – Remote application update support.
- **Not for production** – DevTools automatically disabled when deployed as a JAR.

Example

```
<!-- pom.xml -->

org.springframework.boot
spring-boot-devtools
true <!-- not included in JAR -->
runtime

# application.properties – DevTools configuration
spring.devtools.restart.enabled=true
spring.devtools.livereload.enabled=true
spring.thymeleaf.cache=false # auto-set by DevTools in dev
```

31 How to enable logging in Spring Boot?

Definition

Spring Boot uses Logback by default. Configure via application.properties or logback-spring.xml.

Example

```
# application.properties – basic logging
logging.level.root=INFO
logging.level.com.example=DEBUG
logging.level.org.springframework.web=DEBUG
logging.level.org.hibernate.SQL=DEBUG

logging.file.name=logs/application.log
logging.file.max-size=10MB
logging.file.max-history=30

logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger - %msg%n

// In Java – use SLF4J
@Service
public class OrderService {
private static final Logger log =
LoggerFactory.getLogger(OrderService.class);
// Or with Lombok:
// @Slf4j (Lombok annotation)

public Order create(OrderRequest req) {
log.info("Creating order for customer: {}", req.getCustomerId());
log.debug("Order details: {}", req);
Order order = orderRepo.save(new Order(req));
log.info("Order created: {}", order.getId());
return order;
}
}
```

32 What is spring-boot-starter-data-jpa?

Definition

spring-boot-starter-data-jpa is a starter that bundles Spring Data JPA, Hibernate, and JDBC into one dependency. It auto-configures everything needed for database access.

- **Includes**: spring-data-jpa, hibernate-core, spring-jdbc, HikariCP connection pool.
- **Auto-configures**: DataSource, EntityManagerFactory, TransactionManager, JPA repositories.

Example

```
<!-- Just add this one dependency -->

org.springframework.boot
spring-boot-starter-data-jpa

# application.properties
spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
spring.datasource.username=admin
spring.datasource.password=secret
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

// Entity
@Entity @Table(name="employees")
public class Employee {
@Id @GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
private String name;
private double salary;
}

// Repository – auto-implemented by Spring Data
public interface EmployeeRepository extends JpaRepository {
List findBySalaryGreaterThan(double min);
}
```

33 What is @SpringBootTest? How does it help in testing?

Definition

@SpringBootTest loads the full ApplicationContext for integration testing. It starts the application context just like in production.

Example

```
// Full integration test – loads entire Spring context
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@ActiveProfiles("test")
class OrderControllerIntegrationTest {

@Autowired TestRestTemplate restTemplate;
@Autowired OrderRepository orderRepo;
@MockBean EmailService emailService; // replace real bean with mock

@BeforeEach void setUp() { orderRepo.deleteAll(); }

@Test
void createOrder_returnsCreated() {
CreateOrderRequest req = new CreateOrderRequest("cust-1",
List.of("item-A"));
ResponseEntity res =
restTemplate.postForEntity("/api/orders", req, Order.class);
assertThat(res.getStatusCode()).isEqualTo(HttpStatus.CREATED);
assertThat(orderRepo.count()).isEqualTo(1);
}
}

// src/test/resources/application-test.properties
spring.datasource.url=jdbc:h2:mem:testdb
spring.jpa.hibernate.ddl-auto=create-drop
```

34 What is Flyway and Liquibase in Spring Boot?

Definition

Both are database migration tools that version-control your database schema changes.

- **Flyway**: SQL or Java migrations. Files named V1__description.sql, V2__description.sql. Simple, SQL-first.
- **Liquibase**: XML, YAML, JSON, or SQL changelogs. More complex, better rollback support.

Example

```
<!-- Flyway dependency -->

flyway-core

# application.properties

spring.flyway.enabled=true

spring.flyway.locations=classpath:db/migration

# Migration files in src/main/resources/db/migration/

# V1__create_users.sql

CREATE TABLE users (

id BIGSERIAL PRIMARY KEY,

name VARCHAR(100) NOT NULL,

email VARCHAR(100) UNIQUE NOT NULL

);

# V2__add_phone_to_users.sql

ALTER TABLE users ADD COLUMN phone VARCHAR(20);

# V3__add_roles_table.sql

CREATE TABLE roles ( id BIGSERIAL PRIMARY KEY, name VARCHAR(50) );

# Flyway runs these IN ORDER on startup and tracks what has run
```

35 How to configure Spring Boot for different environments?

Definition

Use Spring Profiles to have different configurations for dev, test, and production.

Example

```
# application.properties (defaults for all profiles)
spring.application.name=my-service
server.port=8080

# application-dev.properties (active in dev)
spring.datasource.url=jdbc:h2:mem:devdb
spring.jpa.show-sql=true
logging.level.com.example=DEBUG

# application-prod.properties (active in prod)
spring.datasource.url=jdbc:postgresql://prod-server:5432/mydb
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASS}
spring.jpa.show-sql=false
logging.level.root=WARN

# Activate profile:
# application.properties: spring.profiles.active=dev
# Environment variable: SPRING_PROFILES_ACTIVE=prod
# Command line: java -jar app.jar --spring.profiles.active=prod

// Profile-specific beans
@Configuration @Profile("dev")
public class DevConfig {
@Bean public DataSource h2DataSource() { ... }
}

@Configuration @Profile("prod")
public class ProdConfig {
@Bean public DataSource postgresDataSource() { ... }
}
```

36 What is the purpose of Spring Profiles?

Definition

Profiles allow you to have different beans and configurations active for different environments (dev/test/prod) without changing code.

Example

```
// @Profile on beans
@Service @Profile("dev")
public class MockPaymentService implements PaymentService {
@Override public void charge(double amt) {
System.out.println("[MOCK] Charged: " + amt);
}
}

@Service @Profile("prod")
public class StripePaymentService implements PaymentService {
@Override public void charge(double amt) {
stripeApi.charge(amt); // real payment
}
}

// Multi-profile: active in dev OR test
@Service @Profile({"dev","test"})
public class InMemoryEmailService implements EmailService { }

// NOT in prod
@Component @Profile("!prod")
public class DataLoader implements CommandLineRunner { }
```

SECTION 4 – REST API, Exceptions & Security

37 How to create a REST API in Spring Boot?

Definition

Use @RestController with HTTP method mapping annotations for a complete REST API.

Example

```
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

@Autowired EmployeeService service;

// GET all
@GetMapping
public ResponseEntity> getAll() {
return ResponseEntity.ok(service.findAll());
}

// GET by id
@GetMapping("/{id}")
public ResponseEntity getById(@PathVariable Long id) {
return service.findById(id)
.map(ResponseEntity::ok)
.orElse(ResponseEntity.notFound().build());
}

// GET with query params
@GetMapping("/search")
public List search(
@RequestParam String dept,
@RequestParam(defaultValue="0") int page) {
return service.findByDept(dept, page);
}

// POST – create
@PostMapping
public ResponseEntity create(
@RequestBody @Valid Employee emp) {
Employee saved = service.save(emp);
URI location = URI.create("/api/employees/" + saved.getId());
return ResponseEntity.created(location).body(saved);
}

// PUT – update
@PutMapping("/{id}")
public ResponseEntity update(
@PathVariable Long id, @RequestBody @Valid Employee emp) {
emp.setId(id);
return ResponseEntity.ok(service.save(emp));
}
```

```
// DELETE
@DeleteMapping("/{id}")
public ResponseEntity delete(@PathVariable Long id) {
service.delete(id);
return ResponseEntity.noContent().build();
}
}
```

38 How to expose and consume REST APIs in Spring Boot?

Definition

Expose APIs using @RestController. Consume external APIs using RestTemplate or WebClient.

Example

```
// CONSUMING external APIs with RestTemplate
@Service
public class WeatherClient {
private final RestTemplate restTemplate;

public WeatherClient(RestTemplateBuilder builder) {
this.restTemplate = builder
.setConnectTimeout(Duration.ofSeconds(5))
.setReadTimeout(Duration.ofSeconds(10))
.build();
}

public WeatherDTO getWeather(String city) {
String url = "https://api.weather.com/v1?city=" + city;
HttpHeaders headers = new HttpHeaders();
headers.set("X-API-Key", "your-api-key");
HttpEntity entity = new HttpEntity<>(headers);
return restTemplate.exchange(
url, HttpMethod.GET, entity, WeatherDTO.class).getBody();
}
}

// CONSUMING with WebClient (reactive/modern)
@Service
public class ProductClient {
private final WebClient webClient;
public ProductClient(WebClient.Builder builder) {
this.webClient = builder.baseUrl("http://product-service").build();
}
public Mono getProduct(Long id) {
return webClient.get().uri("/api/products/{id}", id)
.retrieve()
.bodyToMono(Product.class);
}
}
```

39 How to handle exceptions globally in Spring Boot with @ControllerAdvice?

Definition

@RestControllerAdvice provides global exception handling for all @RestController classes.

Example

```
// Custom exceptions
public class ResourceNotFoundException extends RuntimeException {
public ResourceNotFoundException(String msg) { super(msg); }
}
public class ValidationException extends RuntimeException { ... }

// Error response DTO
record ErrorResponse(int status, String message, LocalDateTime timestamp) {}

// Global exception handler
@RestControllerAdvice
public class GlobalExceptionHandler {

@ExceptionHandler(ResourceNotFoundException.class)
@ResponseStatus(HttpStatus.NOT_FOUND)
public ErrorResponse handleNotFound(ResourceNotFoundException ex) {
return new ErrorResponse(404, ex.getMessage(), LocalDateTime.now());
}

@ExceptionHandler(MethodArgumentNotValidException.class)
@ResponseStatus(HttpStatus.BAD_REQUEST)
public ErrorResponse handleValidation(MethodArgumentNotValidException ex) {
String msg = ex.getBindingResult().getFieldErrors().stream()
.map(e -> e.getField() + ": " + e.getDefaultMessage())
.collect(Collectors.joining(", "));
return new ErrorResponse(400, msg, LocalDateTime.now());
}

@ExceptionHandler(Exception.class)
@ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
public ErrorResponse handleAll(Exception ex) {
return new ErrorResponse(500, "Internal error", LocalDateTime.now());
}
}
```

40 How does Spring Security work? Authentication and Authorisation.

Definition

Spring Security intercepts all HTTP requests via a FilterChain, authenticates the user, and checks if they are authorised to access the requested resource.

- **Authentication**: Who is this user? Validates credentials (username/password, JWT, OAuth2).
- **Authorisation**: What can this user do? Checks roles/permissions after authentication.
- **SecurityFilterChain**: Chain of filters (CSRF, session, authentication, authorisation).

Example

```
// Spring Security 6.x (Spring Boot 3.x)
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
http
.csrf(csrf -> csrf.disable())
.sessionManagement(s -> s
.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
.authorizeHttpRequests(auth -> auth
.requestMatchers("/api/auth/**").permitAll()
.requestMatchers("/api/admin/**").hasRole("ADMIN")
.anyRequest().authenticated()
)
.addFilterBefore(jwtAuthFilter,
UsernamePasswordAuthenticationFilter.class);
return http.build();
}

@Bean
public PasswordEncoder passwordEncoder() {
return new BCryptPasswordEncoder(12);
}
}

// Method-level security
@Service
public class AdminService {
@PreAuthorize("hasRole('ADMIN')")
public List getAllUsers() { ... }

@PreAuthorize("hasRole('ADMIN') or #id == authentication.principal.id")
public User getUser(Long id) { ... }
}
```

41 How to secure Spring Boot microservices with API Gateway and JWT?

Definition

JWT (JSON Web Token) authentication at the API Gateway validates all incoming requests and passes user information to downstream services.

Example

```
// 1. Login – AuthController issues JWT
@PostMapping("/api/auth/login")
public ResponseEntity login(@RequestBody LoginRequest req) {
authManager.authenticate(new UsernamePasswordAuthenticationToken(
req.username(), req.password()));
UserDetails user = userService.loadUserByUsername(req.username());
String token = jwtUtils.generateToken(user);
return ResponseEntity.ok(new JwtResponse(token));
}

// 2. JWT Filter – validates token on every request
@Component
public class JwtAuthFilter extends OncePerRequestFilter {
@Override
protected void doFilterInternal(HttpServletRequest req,
HttpServletResponse res, FilterChain chain)
throws ServletException, IOException {
String header = req.getHeader("Authorization");
if (header != null && header.startsWith("Bearer ")) {
String token = header.substring(7);
if (jwtUtils.isTokenValid(token)) {
String username = jwtUtils.extractUsername(token);
UsernamePasswordAuthenticationToken auth =
new UsernamePasswordAuthenticationToken(
username, null, jwtUtils.getAuthorities(token));
SecurityContextHolder.getContext().setAuthentication(auth);
}
}
chain.doFilter(req, res);
}
}

// 3. API Gateway validates JWT; downstream services trust the gateway
```

42 How to configure multiple databases in Spring Boot?

Definition

Use multiple @DataSource and @EntityManagerFactory beans, one per database.

Example

```
// Primary datasource (auto-configured)
@Primary
@Bean @ConfigurationProperties("spring.datasource.primary")
public DataSource primaryDataSource() {
return DataSourceBuilder.create().build();
}

// Secondary datasource
@Bean @ConfigurationProperties("spring.datasource.secondary")
public DataSource secondaryDataSource() {
return DataSourceBuilder.create().build();
}

# application.properties
spring.datasource.primary.url=jdbc:postgresql://db1:5432/orders
spring.datasource.primary.username=user1

spring.datasource.secondary.url=jdbc:mysql://db2:3306/analytics
spring.datasource.secondary.username=user2

// Primary repository package
@Configuration
@EnableJpaRepositories(
basePackages = "com.example.orders.repository",
entityManagerFactoryRef = "primaryEntityManager",
transactionManagerRef = "primaryTransactionManager")
public class PrimaryDatabaseConfig { ... }
```

43 How to configure rate limiting in Spring Boot microservice?

Definition

Use Bucket4j, Resilience4j, or Spring Cloud Gateway's RateLimiter filter.

Example

```
// With Resilience4j RateLimiter
@Service
public class ApiService {
@RateLimiter(name = "apiLimit", fallbackMethod = "rateLimitFallback")
public String callExternalApi() {
return externalClient.call();
}
public String rateLimitFallback(RequestNotPermitted ex) {
return "Too many requests – please try again later";
}
}
```

```
# application.yml
resilience4j:
ratelimiter:
instances:
apiLimit:
limit-for-period: 10 # 10 requests
limit-refresh-period: 1s # per second
timeout-duration: 0s # don't wait – fail immediately
```

```
// Spring Cloud Gateway – rate limit per route
spring:
cloud:
gateway:
routes:
- id: order-service
uri: lb://order-service
filters:
- name: RequestRateLimiter
args:
redis-rate-limiter.replenishRate: 10
redis-rate-limiter.burstCapacity: 20
```

SECTION 5 – Microservices & Advanced Topics

44 How do you implement microservices using Spring Boot?

Definition

Each microservice is a standalone Spring Boot application with its own database, communicating via REST or messaging.

- **Service Discovery**: Eureka Server + @EnableEurekaClient.
- **API Gateway**: Spring Cloud Gateway routes requests.
- **Config Server**: Spring Cloud Config centralises configuration.
- **Load Balancing**: @LoadBalanced RestTemplate / OpenFeign.
- **Circuit Breaker**: Resilience4j for fault tolerance.
- **Messaging**: Kafka or RabbitMQ for async communication.

Example

```
// Order Service – standalone Spring Boot app
@SpringBootApplication
@EnableFeignClients
public class OrderServiceApp { ... }

// Call Product Service via Feign (service discovery)
@FeignClient(name = "product-service",
fallback = ProductClientFallback.class)
public interface ProductClient {
@GetMapping("/api/products/{id}")
ProductDTO getProduct(@PathVariable Long id);
}

// application.yml
spring:
application:
name: order-service
eureka:
client:
service-url:
defaultZone: http://eureka-server:8761/eureka/
```

45 Circuit Breaker – how to implement in Spring Boot?

Definition

A circuit breaker stops calling a failing service after a threshold and returns a fallback to prevent cascading failures.

Example

```
<!-- pom.xml -->
spring-cloud-starter-circuitbreaker-resilience4j

@Service
public class OrderService {
private final ProductClient productClient;

@CircuitBreaker(name = "productCB", fallbackMethod = "productFallback")
@Retry(name = "productRetry")
@TimeLimiter(name = "productCB")
public CompletableFuture getProduct(Long id) {
return CompletableFuture.supplyAsync(
() -> productClient.getProduct(id));
}

// Fallback – called when circuit is open
public CompletableFuture productFallback(
Long id, Throwable ex) {
log.warn("Product unavailable: {}", ex.getMessage());
return CompletableFuture.completedFuture(
new ProductDTO(id, "Unavailable", 0.0));
}
}

# application.yml
resilience4j:
circuitbreaker:
instances:
productCB:
failure-rate-threshold: 50
wait-duration-in-open-state: 10s
sliding-window-size: 10
```

46 Explain the internal working of Spring AOP

Definition

Spring AOP works by creating proxy objects that wrap the target beans and intercept method calls to apply advice.

- **1. Spring scans for @Aspect classes** at application startup.
- **2. For each target bean**, Spring creates a Proxy (JDK or CGLIB).
- **3. When a method is called**, the Proxy intercepts it.
- **4. Spring evaluates Pointcut expressions** to find matching Advice.
- **5. Advice executes** (@Before, @After, @Around) around the target method.

Example

```
// AOP execution flow:
Client code calls: orderService.placeOrder(req)
|
Proxy intercepts call
|
@Before LoggingAspect.logBefore() -- runs first
@Around TimingAspect.time() -------- wraps method
|
Actual orderService.placeOrder() runs
|
@AfterReturning LoggingAspect.logAfter()
|
Return to caller

// Enabling AOP
@Configuration
@EnableAspectJAutoProxy // activates @Aspect support
@ComponentScan
public class AppConfig { }

// OR: automatic in Spring Boot
// Just add spring-boot-starter-aop dependency
```

47 How to implement security in Spring Boot – overview

Definition

Spring Security provides authentication, authorisation, and protection against common attacks (CSRF, XSS, session fixation).

Example

```
<!-- Dependency -->
spring-boot-starter-security
<!-- Default: all URLs protected, auto login form, random password in logs -->
// Custom security for a REST API (stateless JWT)
@Configuration @EnableWebSecurity
public class SecurityConfig {
@Bean
public SecurityFilterChain chain(HttpSecurity http) throws Exception {
return http
.csrf(c -> c.disable())
.sessionManagement(s -> s
.sessionCreationPolicy(STATELESS))
.authorizeHttpRequests(a -> a
.requestMatchers("/api/auth/**").permitAll()
.requestMatchers("/api/admin/**").hasRole("ADMIN")
.anyRequest().authenticated())
.addFilterBefore(jwtFilter,
UsernamePasswordAuthenticationFilter.class)
.build();
}
@Bean
public PasswordEncoder encoder() { return new BCryptPasswordEncoder(); }
}
// UserDetailsService – load user from DB
@Service
public class UserDetailsServiceImpl implements UserDetailsService {
@Override
public UserDetails loadUserByUsername(String username) {
User user = userRepo.findByUsername(username)
.orElseThrow(() -> new UsernameNotFoundException(username));
return org.springframework.security.core.userdetails.User
.withUsername(user.getUsername())
.password(user.getPassword()) // BCrypt hashed
.roles(user.getRole())
.build();
}
}
```

48 What is Spring Boot DevTools and its benefits?

Definition

DevTools is a developer-productivity module that speeds up development cycle.

- **Automatic Restart** – Detects classpath changes and restarts (2-3x faster than cold start).
- **LiveReload** – Auto-refreshes browser on resource changes.
- **Property Defaults** – Sets cache=false for Thymeleaf, Freemarker etc. automatically.
- **H2 Console** – Enables H2 web console in dev.
- **Remote Dev** – Update remote running app without full redeploy.
- **Automatically disabled in production** – Excluded from fat JAR build.

Example

```
spring-boot-devtools
true

# These are set automatically by DevTools in dev:
spring.thymeleaf.cache=false
spring.freemarker.cache=false
spring.web.resources.cache.period=0
logging.level.web=DEBUG

# Trigger restart: just save any .java file in your IDE
# IntelliJ: Build -> Build Project (Ctrl+F9)
```

49 How to configure multiple databases in Spring Boot?

Definition

Define separate DataSource, EntityManagerFactory, and TransactionManager beans for each database.

Example

```
// Primary DB configuration
@Configuration
@EnableJpaRepositories(
basePackages = "com.example.primary.repo",
entityManagerFactoryRef = "primaryEMF",
transactionManagerRef = "primaryTM")
public class PrimaryDbConfig {
@Primary @Bean
@ConfigurationProperties("spring.datasource.primary")
public DataSource primaryDataSource() {
return DataSourceBuilder.create().build();
}
@Primary @Bean
public LocalContainerEntityManagerFactoryBean primaryEMF(
@Qualifier("primaryDataSource") DataSource ds) {
LocalContainerEntityManagerFactoryBean emf =
new LocalContainerEntityManagerFactoryBean();
emf.setDataSource(ds);
emf.setPackagesToScan("com.example.primary.model");
emf.setJpaVendorAdapter(new HibernateJpaVendorAdapter());
return emf;
}
@Primary @Bean
public PlatformTransactionManager primaryTM(
@Qualifier("primaryEMF") EntityManagerFactory emf) {
return new JpaTransactionManager(emf);
}
}

# application.properties
spring.datasource.primary.url=jdbc:postgresql://host1:5432/db1
spring.datasource.secondary.url=jdbc:mysql://host2:3306/db2
```

50 What is Spring Boot? Summary + How it handles Actuator

Definition

Final summary of Spring Boot and its production-readiness via Actuator.

- **Spring Boot**: Opinionated Spring that auto-configures your application.
- **Actuator**: Production monitoring with /health, /metrics, /info, /loggers, /env endpoints.
- **Custom Health Indicator**: Implement HealthIndicator for custom checks.

Example

```
# Enable Actuator endpoints
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always
management.server.port=8081 # separate port for security

// Custom HealthIndicator
@Component
public class DatabaseHealthIndicator implements HealthIndicator {
@Autowired DataSource dataSource;

@Override
public Health health() {
try (Connection c = dataSource.getConnection()) {
c.isValid(1);
return Health.up()
.withDetail("database", "PostgreSQL")
.withDetail("status", "Connected")
.build();
} catch (Exception e) {
return Health.down(e)
.withDetail("error", e.getMessage())
.build();
}
}
}

// Access: GET /actuator/health
// {"status":"UP","components":{"database":{"status":"UP",
// "details":{"database":"PostgreSQL","status":"Connected"}}}}
```

QUICK REFERENCE – Spring Annotations

Annotation	Category	Purpose
@Component	Bean	Generic Spring-managed bean
@Service	Bean	Business logic layer
@Repository	Bean	Data access; exception translation
@Controller	Bean	MVC controller – returns view name
@RestController	Bean	@Controller + @ResponseBody (returns JSON)
@Autowired	DI	Inject dependency by type
@Qualifier	DI	Choose specific bean by name
@Primary	DI	Default bean when multiple exist
@Value	Config	Inject a property value
@ConfigurationProperties	Config	Bind a property prefix to a POJO
@Bean	Config	Declare a bean in @Configuration class
@Configuration	Config	Marks class as source of @Bean definitions
@SpringBootApplication	Boot	@Configuration + @EnableAutoConfig + @ComponentScan
@EnableAutoConfiguration	Boot	Triggers auto-configuration mechanism
@Profile	Boot	Activate bean only for specified profile
@Transactional	Data	Wrap method/class in a DB transaction
@Entity	JPA	Maps Java class to database table
@Id	JPA	Marks primary key field
@Scope	Bean	Set bean scope (singleton, prototype, request)
@PostConstruct	Lifecycle	Runs after bean initialisation
@PreDestroy	Lifecycle	Runs before bean destruction
@Aspect	AOP	Marks class as an Aspect
@Before/@After/@Around	AOP	Advice type annotations
@ExceptionHandler	Error	Handle specific exception in @Controller
@ControllerAdvice	Error	Global exception handling
@CrossOrigin	Web	Enable CORS for a controller
@GetMapping/@PostMapping etc.	Web	HTTP method mapping shortcuts
@PathVariable	Web	Extract URL path segment
@RequestParam	Web	Extract query parameter
@RequestBody	Web	Deserialise JSON request body
@Valid	Web	Trigger Bean Validation

Annotation	Category	Purpose
@SpringBootTest	Test	Load full context for integration tests
@MockBean	Test	Replace real bean with Mockito mock
@PreAuthorize	Security	Method-level security check (SpEL)